Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Kubernetes Fundamentals for Beginners

Course Code: C742

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	2 October 20265 September 2026	Initial release — Kubernetes Fundamentals for Beginners (C742) Learner Guide covering all 16 labs across five topics.	Dr. Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Introduction **4**
Course Learning Outcomes **4**
Before You Start — Environment Setup **4**
Topic 01 — Kubernetes Fundamentals and Architecture (46%) **5**
Lab 1 — Bootstrap a Kubernetes Cluster and Explore the Architecture 6
Lab 2 — Work with the Kubernetes API and kubectl 7
Lab 3 — Deploy and Manage Pods, ReplicaSets and Deployments 9
Lab 4 — Control Pod Scheduling with Requests, Taints and Affinity 10
Topic 02 — Container Orchestration and Kubernetes Operations (22%) **12**
Lab 5 — Build and Run Containers with Docker and the OCI Standard 12
Lab 6 — Expose Applications with Services and Ingress 14
Lab 7 — Secure the Cluster with RBAC, Secrets and NetworkPolicy 15
Lab 8 — Persist Data with PersistentVolumes, StatefulSets and Storage Classes 17
Topic 03 — Cloud Native Architecture and Scalability (16%) **18**
Lab 9 — Autoscale Workloads with the Horizontal Pod Autoscaler 19
Lab 10 — Build Resilience with Probes, Disruption Budgets and Multi-Tier Architecture 21
Lab 11 — Explore Serverless, Open Standards and the CNCF Landscape 22
Topic 04 — Kubernetes Monitoring and Observability (8%) **24**
Lab 12 — Instrument and Collect Metrics with Prometheus 24
Lab 13 — Apply the Golden Signals, Alerting and Cost Management 26
Topic 05 — Kubernetes Application Deployment and Management (8%) **28**
Lab 14 — Perform Zero-Downtime Rolling Updates and Canary Releases 28
Lab 15 — Package Applications with Helm and Kustomize 30
Lab 16 — Automate Delivery with GitOps and CI/CD, and Conduct a System Review 31
Exam Focus — Cross-Cutting Topics **33**
Revision Tips **34**
Glossary **34**

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Introduction

This Learner Guide accompanies the course Kubernetes Fundamentals for Beginners (C742), conducted by Tertiary Infotech Academy Pte Ltd. It provides step-by-step instructions for all 16 hands-on labs, organised by the five course topics. Every topic maps to a course Learning Outcome and to one of the five CNCF Kubernetes and Cloud Native Associate (KCNA) exam domains, so the course also prepares you for the KCNA certification.

Use this guide alongside the course slides and the lab folders in the labs/ directory of the course repository. Each lab has its own folder containing every YAML manifest, mock data file and helper script it needs, so you can run each lab end to end without writing manifests from scratch. All labs run on a local single-node minikube cluster, so nothing you do here affects a production system.

Course Learning Outcomes

- LO1: Develop an architectural proof of concept for Kubernetes architecture and components.
- LO2: Develop solution architecture by applying container orchestration tools, techniques, and system modelling methods.
- LO3: Identify technical requirements and prepare a technical blueprint for Cloud Native Architecture.
- LO4: Demonstrate how Cloud Native Observability solutions and components address a specific business problem effectively.
- LO5: Implement system reviews to monitor solution status and adapt according to Cloud Native frameworks.

Before You Start — Environment Setup

What you need

- Docker Desktop (macOS/Windows) or Docker Engine (Linux) — used to build container images in Topic 2.
- minikube — runs the local single-node Kubernetes cluster used by every lab.
- kubectl — the command-line client you use to talk to the Kubernetes API throughout the course.
- Helm 3 — used for the packaging lab in Topic 5.
- A code editor such as VS Code, ideally with the YAML and Kubernetes extensions.
- At least 4 GB of free RAM and 2 CPU cores available to minikube.

Start and verify your cluster

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Every lab assumes a running minikube cluster and a working kubectl context. Start the cluster once at the beginning of the course and verify it before each topic. If a lab misbehaves, verifying the cluster is always the first troubleshooting step.

```
$ minikube start --nodes=1 --cpus=2 --memory=4096 --kubernetes-
  version=v1.31.0
$ kubectl version --client        # client is installed
$ kubectl get nodes               # node should report STATUS=Ready
$ kubectl get pods -n kube-system # control-plane components should be
  Running
$ kubectl apply -f labs/lab-02-kubernetes-api/namespace.yaml   # the
  shared kcna-lab namespace
```

Conventions used in every lab

- Every lab has its own folder: labs/lab-NN-<slug>/ containing its YAML manifests, mock data and README.
- Run the commands from inside that lab folder, so relative paths such as -f deployment.yaml resolve.
- Unless a lab says otherwise, all objects are created in the kcna-lab namespace created in Lab 2.
- Manifests are applied declaratively with kubectl apply -f; imperative commands are used only to illustrate a contrast.
- Each lab ends with a 'Test it' check — do not move on until that check passes.
- Clean up with kubectl delete -f . at the end of a lab if you are short of cluster resources.

Topic 01 — Kubernetes Fundamentals and Architecture (46%)

Kubernetes Resources · Architecture · The API · Containers · Scheduling

Key concepts

- Kubernetes is a declarative system: you submit the desired state as an object, and controllers work continuously to make the actual state match it.
- The control plane (kube-apiserver, etcd, kube-scheduler, kube-controller-manager) decides what should run; the worker nodes (kubelet, kube-proxy, container runtime) actually run it.
- The kube-apiserver is the single front door — every component, including kubectl, talks only to the API server, never directly to etcd or to each other.
- etcd is the only stateful component: a consistent, highly available key-value store holding the entire cluster state. Losing etcd means losing the cluster.
- A Pod is the smallest deployable unit — one or more containers sharing a network namespace, an IP address and storage volumes.
- Workload resources build on Pods: a ReplicaSet keeps N replicas alive, a Deployment manages ReplicaSets to give rolling updates and rollback.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- DaemonSets run one Pod per node (agents), StatefulSets give stable identity and storage (databases), Jobs and CronJobs run work to completion.
- Namespaces partition a cluster logically; labels and selectors are how every controller and Service finds the objects it manages.
- The scheduler places a Pod on a node by filtering (which nodes *can* run it) then scoring (which node is *best*), honouring resource requests, taints, tolerations and affinity.
- Resource requests drive scheduling and limits drive enforcement — a container exceeding its memory limit is OOM-killed; exceeding its CPU limit is throttled.

Lab 1 — Bootstrap a Kubernetes Cluster and Explore the Architecture

Objective: Explain the Kubernetes control-plane and node components and verify each one in a running cluster.

Goal: The learner starts a local Kubernetes cluster with minikube, inspects every control-plane and node component, and maps what they observe onto the reference architecture diagram used in the proof of concept.

What you'll build

A running single-node Kubernetes cluster plus an annotated architecture inventory of its components (Tools: minikube, kubectl, kube-apiserver, etcd, kube-scheduler, kube-controller-manager, kubelet, kube-proxy.)

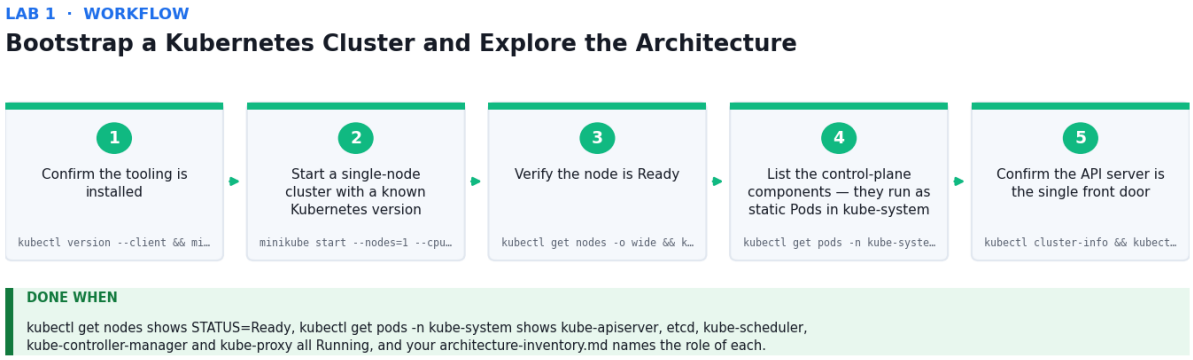


Figure 1: the Bootstrap a Kubernetes Cluster and Explore the Architecture workflow — the first five stages and the verification criterion.

Step-by-step

1. Confirm the tooling is installed and check the client version

```
kubectl version --client && minikube version
```

2. Start a single-node cluster with a known Kubernetes version

```
minikube start --nodes=1 --cpus=2 --memory=4096 --kubernetes-
  version=v1.31.0
```

3. Verify the node is Ready and inspect its capacity and conditions

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
kubectl get nodes -o wide && kubectl describe node minikube | head -40
```

4. List the control-plane components — they run as static Pods in kube-system

```
kubectl get pods -n kube-system -o wide
```

5. Confirm the API server is the single front door and check cluster health

```
kubectl cluster-info && kubectl get --raw='/readyz?verbose' | head -20
```

6. Inspect the etcd static Pod manifest to see how the datastore is configured

```
kubectl -n kube-system describe pod -l component=etcd | head -30
```

7. Watch the scheduler and controller-manager leader election in the logs

```
kubectl -n kube-system logs -l component=kube-scheduler --tail=20
```

8. Inspect kubelet on the node — the agent that actually starts containers

```
minikube ssh -- 'sudo systemctl status kubelet --no-pager | head -15'
```

9. List every API resource the cluster serves, grouped by API group

```
kubectl api-resources --sort-by=name | head -40
```

10. Complete architecture-inventory.md, recording each component and its role

```
bash cluster-info.sh > my-cluster-inventory.txt
```

Test it

kubectl get nodes shows STATUS=Ready, kubectl get pods -n kube-system shows kube-apiserver, etcd, kube-scheduler, kube-controller-manager and kube-proxy all Running, and your architecture-inventory.md names the role of each.

Note: All manifests, mock data and the full command list for this lab are in labs/lab-01-*/ . Run the commands from inside that folder.

Lab 2 — Work with the Kubernetes API and kubectl

Objective: Use the declarative Kubernetes API to create, inspect and delete objects, and read the API directly.

Goal: The learner drives the Kubernetes API through kubectl and through raw HTTP, comparing imperative and declarative object management, and reads the anatomy of an API object — apiVersion, kind, metadata, spec and status.

What you'll build

A namespace and Pod created declaratively, plus a raw API call transcript showing the same objects (Tools: kubectl, kube-apiserver, kubectl proxy, YAML manifests, API groups and versions.)

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

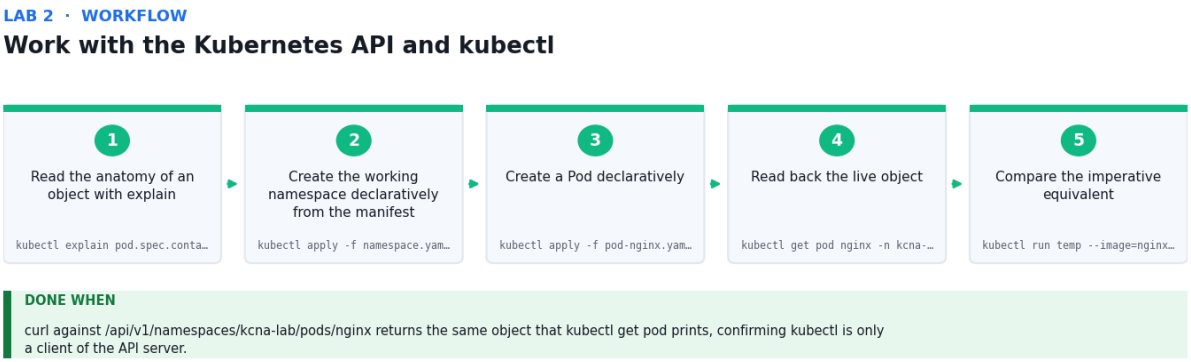


Figure 2: the Work with the Kubernetes API and kubectl workflow — the first five stages and the verification criterion.

Step-by-step

1. Read the anatomy of an object with explain, field by field

```
kubectl explain pod.spec.containers --recursive | head -30
```

2. Create the working namespace declaratively from the manifest

```
kubectl apply -f namespace.yaml
```

3. Create a Pod declaratively and watch it move through its phases

```
kubectl apply -f pod-nginx.yaml && kubectl get pod -n kcna-lab -w --
  timeout=60s
```

4. Read back the live object, including the status the controllers wrote

```
kubectl get pod nginx -n kcna-lab -o yaml | head -40
```

5. Compare the imperative equivalent, using --dry-run to see the generated object

```
kubectl run temp --image=nginx:1.27 --dry-run=client -o yaml
```

6. Open a local proxy to the API server and call the REST endpoint directly

```
kubectl proxy --port=8001 &
```

7. Query the same Pod over raw HTTP to prove kubectl is only an API client

```
curl -s http://127.0.0.1:8001/api/v1/namespaces/kcna-lab/pods/nginx |
  head -30
```

8. Inspect the API groups and versions the server exposes

```
curl -s http://127.0.0.1:8001/apis | head -30
```

9. Use a label selector and JSONPath to extract just the fields you need

```
kubectl get pods -n kcna-lab -l app=nginx -o
  jsonpath='{.items[*].status.podIP}'
```

10. Clean up the proxy process when you are finished

```
kill %1
```

Test it

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

curl against /api/v1/namespaces/kcna-lab/pods/nginx returns the same object that kubectl get pod prints, confirming kubectl is only a client of the API server.

Note: All manifests, mock data and the full command list for this lab are in labs/lab-02-*/ . Run the commands from inside that folder.

Lab 3 — Deploy and Manage Pods, ReplicaSets and Deployments

Objective: Build the core workload resources and explain how a Deployment manages ReplicaSets which manage Pods.

Goal: The learner deploys the sample store-front application, scales it, breaks it deliberately to observe self-healing, then performs a rolling update and a rollback to see the Deployment/ReplicaSet/Pod ownership chain in action.

What you'll build

A 3-replica Deployment that survives Pod deletion, plus a completed rolling update and rollback (Tools: Deployment, ReplicaSet, Pod, kubectl scale, kubectl rollout, labels and selectors.)

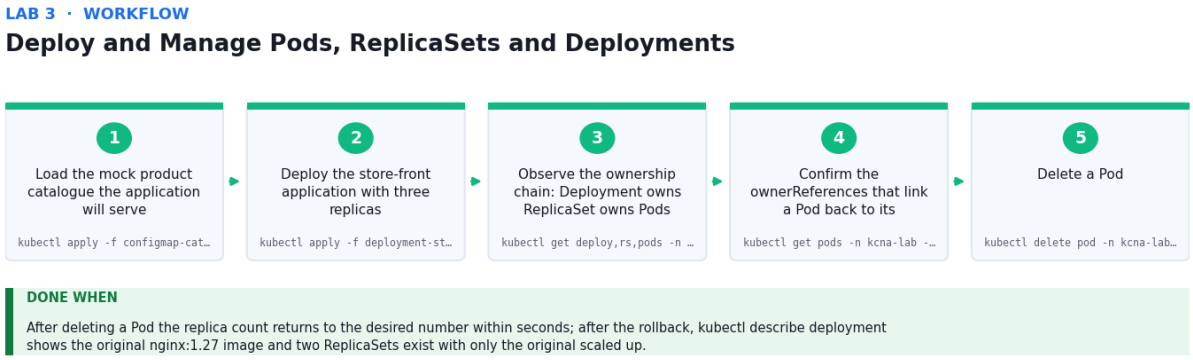


Figure 3: the Deploy and Manage Pods, ReplicaSets and Deployments workflow — the first five stages and the verification criterion.

Step-by-step

1. Load the mock product catalogue the application will serve

   ```
   kubectl apply -f configmap-catalog.yaml
   ```
2. Deploy the store-front application with three replicas

   ```
   kubectl apply -f deployment-storefront.yaml
   ```
3. Observe the ownership chain: Deployment owns ReplicaSet owns Pods

   ```
   kubectl get deploy,rs,pods -n kcna-lab -l app=storefront
   ```
4. Confirm the ownerReferences that link a Pod back to its ReplicaSet

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
kubectl get pods -n kcna-lab -l app=storefront -o
  jsonpath='{.items[0].metadata.ownerReferences[0].kind}'
```

5. Delete a Pod and watch the ReplicaSet controller replace it (self-healing)

```
kubectl delete pod -n kcna-lab -l app=storefront --field-
  selector=status.phase=Running --wait=false
```

6. Scale the Deployment up and confirm the new replica count

```
kubectl scale deployment storefront -n kcna-lab --replicas=5
```

7. Perform a rolling update to a new image version

```
kubectl set image deployment/storefront storefront=nginx:1.27-alpine -n
  kcna-lab
```

8. Watch the rollout progress and see a second ReplicaSet appear

```
kubectl rollout status deployment/storefront -n kcna-lab && kubectl get
  rs -n kcna-lab
```

9. Review the revision history the Deployment keeps for you

```
kubectl rollout history deployment/storefront -n kcna-lab
```

10. Roll back to the previous revision and confirm the image reverted

```
kubectl rollout undo deployment/storefront -n kcna-lab
```

Test it

After deleting a Pod the replica count returns to the desired number within seconds; after the rollback, kubectl describe deployment shows the original nginx:1.27 image and two ReplicaSets exist with only the original scaled up.

Note: All manifests, mock data and the full command list for this lab are in labs/lab-03-*/ . Run the commands from inside that folder.

Lab 4 — Control Pod Scheduling with Requests, Taints and Affinity

Objective: Influence where the scheduler places a Pod using resource requests, node selectors, taints, tolerations and affinity.

Goal: The learner works through the scheduler's filter-and-score decision: setting resource requests that make a Pod unschedulable, labelling nodes, applying taints and tolerations, and expressing node and Pod affinity rules.

What you'll build

A set of Pods demonstrating successful placement, a deliberate Pending Pod, and a toleration that restores scheduling (Tools: kube-scheduler, resource requests and limits, nodeSelector, taints and tolerations, node affinity, pod anti-affinity.)

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

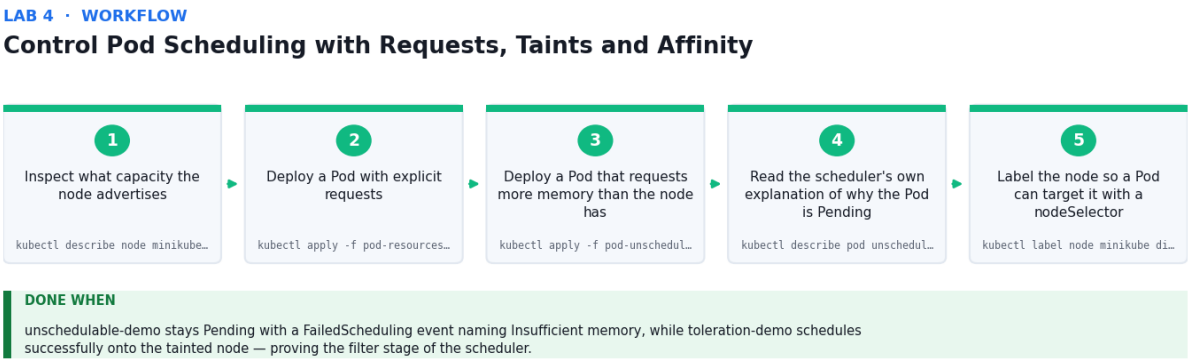


Figure 4: the Control Pod Scheduling with Requests, Taints and Affinity workflow — the first five stages and the verification criterion.

Step-by-step

1. Inspect what capacity the node advertises and what is already requested

```
kubectl describe node minikube | grep -A8 'Allocated resources'
```

2. Deploy a Pod with explicit requests and limits and confirm placement

```
kubectl apply -f pod-resources.yaml && kubectl get pod resources-demo -n
  kcna-lab -o wide
```

3. Deploy a Pod that requests more memory than the node has

```
kubectl apply -f pod-unschedulable.yaml
```

4. Read the scheduler's own explanation of why the Pod is Pending

```
kubectl describe pod unschedulable-demo -n kcna-lab | grep -A5 Events
```

5. Label the node so a Pod can target it with a nodeSelector

```
kubectl label node minikube disktype=ssd --overwrite
```

6. Taint the node so that only tolerating Pods may be scheduled onto it

```
kubectl taint node minikube workload=restricted:NoSchedule --overwrite
```

7. Deploy a Pod carrying the matching toleration and confirm it schedules

```
kubectl apply -f pod-toleration.yaml && kubectl get pod toleration-demo
  -n kcna-lab -o wide
```

8. Deploy a Deployment using pod anti-affinity to spread replicas apart

```
kubectl apply -f deployment-antiaffinity.yaml
```

9. Observe how anti-affinity affects placement on a single-node cluster

```
kubectl get pods -n kcna-lab -l app=spread -o wide
```

10. Remove the taint to restore normal scheduling before the next lab

```
kubectl taint node minikube workload=restricted:NoSchedule-
```

Test it

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

unschedulable-demo stays Pending with a FailedScheduling event naming Insufficient memory, while toleration-demo schedules successfully onto the tainted node — proving the filter stage of the scheduler.

Note: All manifests, mock data and the full command list for this lab are in labs/lab-04-*/ . Run the commands from inside that folder.

Topic 02 — Container Orchestration and Kubernetes Operations (22%)

Orchestration Fundamentals · Runtime · Security · Networking · Service Mesh · Storage

Key concepts

- A container is a process isolated by Linux namespaces and constrained by cgroups — it is not a virtual machine and shares the host kernel.
- Orchestration adds what a single container runtime cannot: scheduling across many hosts, self-healing, service discovery, scaling and rolling updates.
- The Container Runtime Interface (CRI) lets Kubernetes run containerd or CRI-O interchangeably; the OCI standard defines the image and runtime formats.
- The Kubernetes network model requires that every Pod gets its own IP and that all Pods can reach each other without NAT — the CNI plugin implements this.
- A Service gives a stable virtual IP and DNS name in front of an ephemeral set of Pods: ClusterIP (internal), NodePort, LoadBalancer, and Ingress for HTTP routing.
- NetworkPolicies are the Pod-level firewall — by default all traffic is allowed, and a policy switches the selected Pods to default-deny.
- Security is layered as the 4C model: Cloud, Cluster, Container, Code. RBAC, ServiceAccounts, Secrets and Pod Security Standards each cover a different layer.
- RBAC binds a subject (user, group or ServiceAccount) to a Role (namespaced) or ClusterRole (cluster-wide) through a RoleBinding or ClusterRoleBinding.
- A service mesh (Istio, Linkerd) adds mTLS, retries, traffic splitting and per-request telemetry through sidecar proxies, without changing application code.
- Storage is decoupled: a PersistentVolumeClaim requests storage, a StorageClass provisions a PersistentVolume dynamically, and the CSI driver does the work.

Lab 5 — Build and Run Containers with Docker and the OCI Standard

Objective: Build an OCI image, run it as a container, and explain the runtime layer beneath Kubernetes.

Goal: The learner containerises the sample inventory API, inspects the resulting OCI image layers, runs it locally with resource constraints, and then confirms which CRI runtime the Kubernetes cluster is actually using.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

What you'll build

A tagged OCI image of the inventory API, running locally and loaded into the cluster (Tools: Docker, OCI image spec, containerd, CRI, crictl, image layers and registries.)

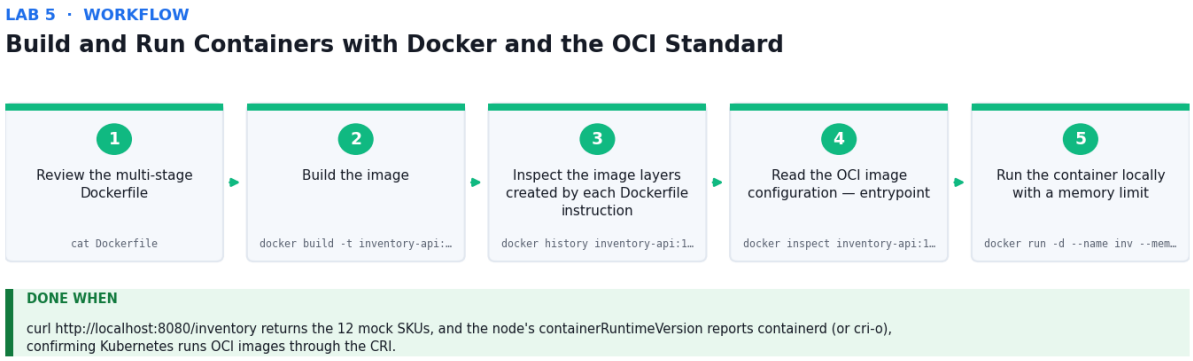


Figure 5: the Build and Run Containers with Docker and the OCI Standard workflow — the first five stages and the verification criterion.

Step-by-step

1. Review the multi-stage Dockerfile and note why the runtime stage is slim

   ```
   cat Dockerfile
   ```
2. Build the image and tag it with a version

   ```
   docker build -t inventory-api:1.0 .
   ```
3. Inspect the image layers created by each Dockerfile instruction

   ```
   docker history inventory-api:1.0
   ```
4. Read the OCI image configuration — entrypoint, env and architecture

   ```
   docker inspect inventory-api:1.0 --format '{{json .Config}}' | python3
     -m json.tool | head -20
   ```
5. Run the container locally with a memory limit and a published port

   ```
   docker run -d --name inv --memory=256m -p 8080:8080 inventory-api:1.0
   ```
6. Call the API and confirm the mock inventory data is served

   ```
   curl -s http://localhost:8080/inventory | head -20
   ```
7. Show that the container is just an isolated process on the host

   ```
   docker top inv && docker stats --no-stream inv
   ```
8. Stop and remove the local container

   ```
   docker rm -f inv
   ```
9. Load the image into the cluster so Kubernetes can run it without a registry

   ```
   minikube image load inventory-api:1.0
   ```
10. Confirm which CRI runtime the node uses to run containers

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
kubectl get node minikube -o
  jsonpath='{.status.nodeInfo.containerRuntimeVersion}'
```

Test it

curl http://localhost:8080/inventory returns the 12 mock SKUs, and the node's containerRuntimeVersion reports containerd (or cri-o), confirming Kubernetes runs OCI images through the CRI.

Note: All manifests, mock data and the full command list for this lab are in labs/lab-05-*/ . Run the commands from inside that folder.

Lab 6 — Expose Applications with Services and Ingress

Objective: Apply Kubernetes networking to give stable discovery and external access to ephemeral Pods.

Goal: The learner deploys the inventory API, fronts it with a ClusterIP Service, proves DNS-based service discovery from inside the cluster, then exposes it externally through NodePort and an Ingress with host-based routing.

What you'll build

An internally discoverable ClusterIP Service and an externally reachable Ingress route (Tools: Service (ClusterIP/NodePort/LoadBalancer), CoreDNS, Endpoints, Ingress, ingress-nginx, CNI.)

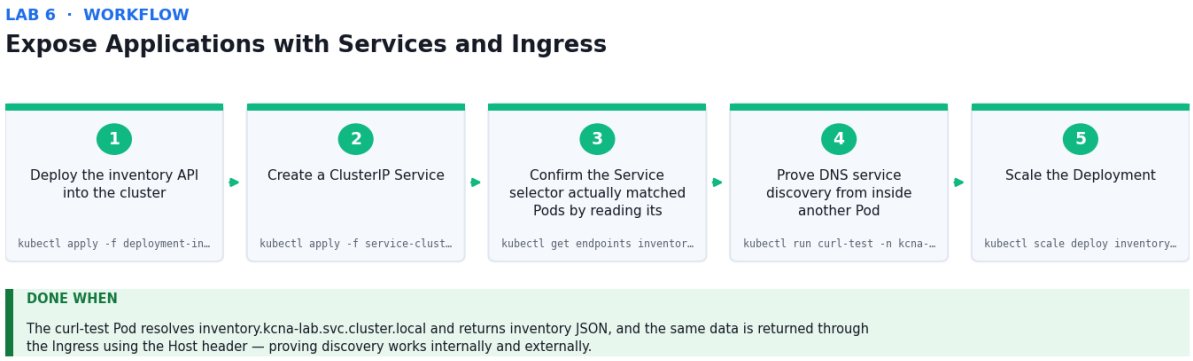


Figure 6: the Expose Applications with Services and Ingress workflow — the first five stages and the verification criterion.

Step-by-step

1. Deploy the inventory API into the cluster

```
kubectl apply -f deployment-inventory.yaml
```

2. Create a ClusterIP Service and inspect the virtual IP it receives

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
kubectl apply -f service-clusterip.yaml && kubectl get svc inventory -n
  kcna-lab
```

3. Confirm the Service selector actually matched Pods by reading its Endpoints

```
kubectl get endpoints inventory -n kcna-lab -o wide
```

4. Prove DNS service discovery from inside another Pod

```
kubectl run curl-test -n kcna-lab --rm -it --image=curlimages/curl --
  restart=Never -- curl -s
  http://inventory.kcna-lab.svc.cluster.local/inventory
```

5. Scale the Deployment and watch the Endpoints list grow automatically

```
kubectl scale deploy inventory -n kcna-lab --replicas=3 && kubectl get
  endpoints inventory -n kcna-lab
```

6. Expose the same Pods externally with a NodePort Service

```
kubectl apply -f service-nodeport.yaml
```

7. Reach the NodePort from outside the cluster

```
curl -s $(minikube ip):30080/inventory | head -10
```

8. Enable the ingress controller that will terminate HTTP routing

```
minikube addons enable ingress
```

9. Create the Ingress with host-based routing to the Service

```
kubectl apply -f ingress-inventory.yaml && kubectl get ingress -n kcna-
  lab
```

10. Test the host-based route through the ingress controller

```
curl -s -H 'Host: inventory.kcna.local' http://$(minikube ip)/inventory
  | head -10
```

Test it

The curl-test Pod resolves inventory.kcna-lab.svc.cluster.local and returns inventory JSON, and the same data is returned through the Ingress using the Host header — proving discovery works internally and externally.

Note: All manifests, mock data and the full command list for this lab are in labs/lab-06-*/ . Run the commands from inside that folder.

Lab 7 — Secure the Cluster with RBAC, Secrets and NetworkPolicy

Objective: Apply the 4C security model using RBAC, ServiceAccounts, Secrets and network segmentation.

Goal: The learner creates a least-privilege ServiceAccount bound to a namespaced Role, verifies the permission boundary with kubectl auth can-i, mounts credentials from a Secret, and switches the namespace to default-deny with a NetworkPolicy.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

What you'll build

A least-privilege ServiceAccount, a mounted Secret, and a default-deny network posture with one allowed path (Tools: RBAC (Role, RoleBinding), ServiceAccount, Secret, NetworkPolicy, Pod Security Standards.)

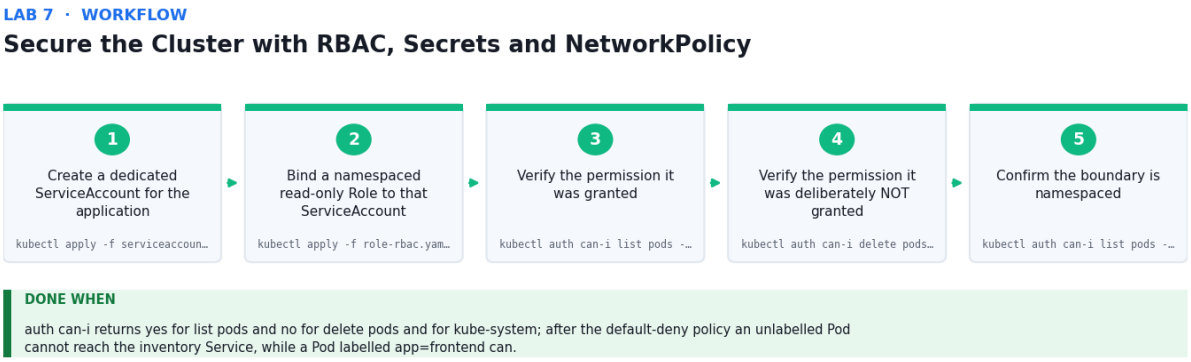


Figure 7: the Secure the Cluster with RBAC, Secrets and NetworkPolicy workflow — the first five stages and the verification criterion.

Step-by-step

1. Create a dedicated ServiceAccount for the application

   ```
   kubectl apply -f serviceaccount.yaml
   ```
2. Bind a namespaced read-only Role to that ServiceAccount

   ```
   kubectl apply -f role-rbac.yaml
   ```
3. Verify the permission it was granted

   ```
   kubectl auth can-i list pods -n kcna-lab --
     as=system:serviceaccount:kcna-lab:inventory-reader
   ```
4. Verify the permission it was deliberately NOT granted

   ```
   kubectl auth can-i delete pods -n kcna-lab --
     as=system:serviceaccount:kcna-lab:inventory-reader
   ```
5. Confirm the boundary is namespaced, not cluster-wide

   ```
   kubectl auth can-i list pods -n kube-system --
     as=system:serviceaccount:kcna-lab:inventory-reader
   ```
6. Create the mock database credentials as a Secret

   ```
   kubectl apply -f secret-dbcreds.yaml
   ```
7. Show that a Secret is base64-encoded, not encrypted at rest by default

   ```
   kubectl get secret db-credentials -n kcna-lab -o
     jsonpath='{.data.password}' | base64 -d
   ```
8. Run a Pod that consumes the Secret as environment variables

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
kubectl apply -f pod-secret-consumer.yaml && kubectl logs secret-
  consumer -n kcna-lab
```

9. Apply a default-deny NetworkPolicy and confirm traffic is now blocked

```
kubectl apply -f networkpolicy-default-deny.yaml
```

10. Allow only the frontend to reach the API, then re-test connectivity

```
kubectl apply -f networkpolicy-allow-frontend.yaml
```

Test it

auth can-i returns yes for list pods and no for delete pods and for kube-system; after the default-deny policy an unlabelled Pod cannot reach the inventory Service, while a Pod labelled app=frontend can.

Note: All manifests, mock data and the full command list for this lab are in labs/lab-07-*/ . Run the commands from inside that folder.

Lab 8 — Persist Data with PersistentVolumes, StatefulSets and Storage Classes

Objective: Decouple storage from compute using PVCs, StorageClasses and StatefulSets, and model a service mesh's role.

Goal: The learner provisions storage dynamically through a StorageClass, proves data survives Pod deletion, deploys a StatefulSet with per-replica volumes and stable identities, and compares this with the sidecar model used by a service mesh.

What you'll build

A dynamically provisioned PVC whose data survives Pod deletion, plus a 3-replica StatefulSet with stable storage (Tools: PersistentVolume, PersistentVolumeClaim, StorageClass, CSI, StatefulSet, headless Service, service mesh concepts.)

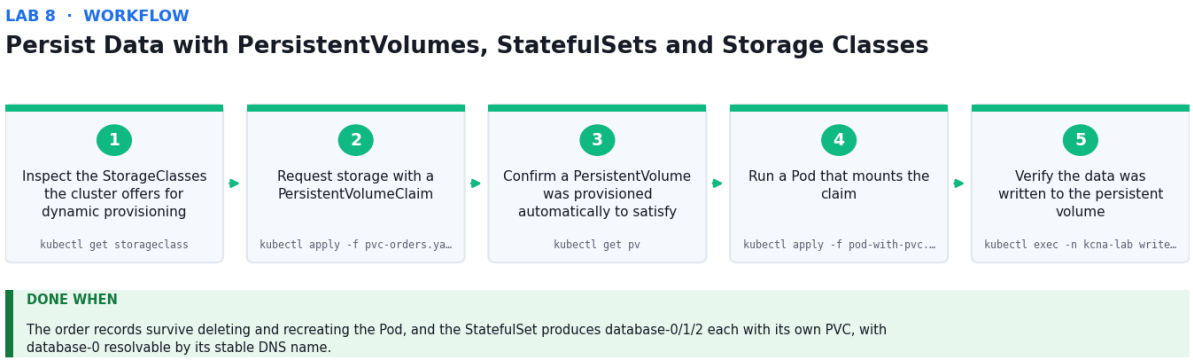


Figure 8: the Persist Data with PersistentVolumes, StatefulSets and Storage Classes workflow — the first five stages and the verification criterion.

Step-by-step

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

1. Inspect the StorageClasses the cluster offers for dynamic provisioning

```
kubectl get storageclass
```

2. Request storage with a PersistentVolumeClaim

```
kubectl apply -f pvc-orders.yaml && kubectl get pvc -n kcna-lab
```

3. Confirm a PersistentVolume was provisioned automatically to satisfy it

```
kubectl get pv
```

4. Run a Pod that mounts the claim and writes the mock order records

```
kubectl apply -f pod-with-pvc.yaml
```

5. Verify the data was written to the persistent volume

```
kubectl exec -n kcna-lab writer -- cat /data/orders.csv | head -5
```

6. Delete the Pod — the volume and its data must survive

```
kubectl delete pod writer -n kcna-lab
```

7. Recreate the Pod and confirm the data is still there

```
kubectl apply -f pod-with-pvc.yaml && kubectl exec -n kcna-lab writer --
  wc -l /data/orders.csv
```

8. Deploy a StatefulSet with a headless Service for stable identities

```
kubectl apply -f service-headless.yaml -f statefulset-database.yaml
```

9. Observe ordinal Pod names and the PVC created per replica

```
kubectl get pods,pvc -n kcna-lab -l app=database
```

10. Resolve a single replica by its stable DNS name

```
kubectl run dns-test -n kcna-lab --rm -it --image=busybox:1.36 --
  restart=Never -- nslookup database-0.database.kcna-
  lab.svc.cluster.local
```

Test it

The order records survive deleting and recreating the Pod, and the StatefulSet produces database-0/1/2 each with its own PVC, with database-0 resolvable by its stable DNS name.

Note: All manifests, mock data and the full command list for this lab are in labs/lab-08-*/ . Run the commands from inside that folder.

Topic 03 — Cloud Native Architecture and Scalability (16%)

Autoscaling · Serverless · Community & Governance · Roles & Personas · Open Standards

Key concepts

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Cloud native means building loosely-coupled, resilient, observable and manageable systems that exploit elastic infrastructure — not simply running old software in containers.
- The microservices trade-off: independent deploy and scale, at the cost of network latency, distributed failure modes and far greater operational complexity.
- Kubernetes autoscales on three axes: the HPA adds Pod replicas, the VPA right-sizes requests and limits, and the Cluster Autoscaler adds or removes nodes.
- The HPA computes desired replicas as ceil(current replicas x current metric / target metric), so target selection directly determines cost and headroom.
- Serverless removes server management and scales to zero; Knative and KEDA bring the event-driven, scale-to-zero model onto Kubernetes itself.
- Resilience is designed in: liveness probes restart a hung container, readiness probes remove a Pod from the Service, and a PodDisruptionBudget protects availability during maintenance.
- The CNCF stewards the ecosystem with a maturity ladder — Sandbox, Incubating, Graduated — that is a genuine signal of production readiness.
- Open standards prevent lock-in: OCI for images, CRI for runtimes, CNI for networking, CSI for storage, SMI for meshes and OpenTelemetry for telemetry.
- Cloud native roles and personas — platform engineer, SRE, DevOps engineer, security engineer — divide responsibility across the same shared platform.
- The technical blueprint records the decisions: chosen components, their interfaces, non-functional requirements, and the rationale behind each trade-off.

Lab 9 — Autoscale Workloads with the Horizontal Pod Autoscaler

Objective: Identify the technical requirements for elastic scaling and demonstrate autoscaling under real load.

Goal: The learner enables the metrics pipeline, deploys a CPU-bound service with the resource requests the HPA needs, applies an HPA, generates load, and records the scale-up and scale-down behaviour as evidence for the blueprint.

What you'll build

A working HPA that scales a Deployment from 1 to 10 replicas under load and back down again (Tools: HorizontalPodAutoscaler, metrics-server, resource requests, Deployment, Cluster Autoscaler concepts.)

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

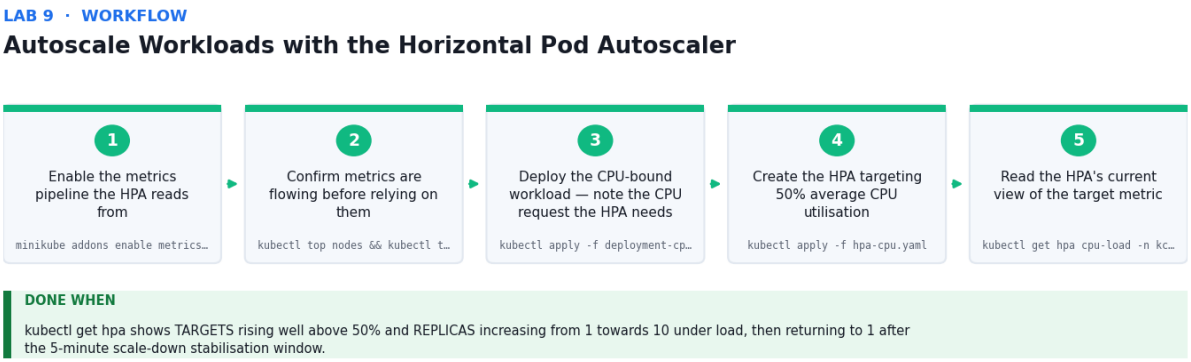


Figure 9: the Autoscale Workloads with the Horizontal Pod Autoscaler workflow — the first five stages and the verification criterion.

Step-by-step

1. Enable the metrics pipeline the HPA reads from

   ```
   minikube addons enable metrics-server
   ```
2. Confirm metrics are flowing before relying on them

   ```
   kubectl top nodes && kubectl top pods -n kcna-lab
   ```
3. Deploy the CPU-bound workload — note the CPU request the HPA needs

   ```
   kubectl apply -f deployment-cpu-load.yaml -f service-cpu-load.yaml
   ```
4. Create the HPA targeting 50% average CPU utilisation

   ```
   kubectl apply -f hpa-cpu.yaml
   ```
5. Read the HPA's current view of the target metric

   ```
   kubectl get hpa cpu-load -n kcna-lab -o wide
   ```
6. Start the load generator to drive CPU above the target

   ```
   kubectl apply -f load-generator.yaml
   ```
7. Watch the HPA scale the Deployment up in response

   ```
   kubectl get hpa cpu-load -n kcna-lab -w --timeout=300s
   ```
8. Record the scaling decisions the controller made and why

   ```
   kubectl describe hpa cpu-load -n kcna-lab | grep -A10 Events
   ```
9. Remove the load and observe the stabilisation window before scale-down

   ```
   kubectl delete -f load-generator.yaml
   ```
10. Complete scaling-observations.md with your replica-count timeline

    ```
    kubectl get hpa cpu-load -n kcna-lab
    ```

Test it

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

kubectl get hpa shows TARGETS rising well above 50% and REPLICAS increasing from 1 towards 10 under load, then returning to 1 after the 5-minute scale-down stabilisation window.

Note: All manifests, mock data and the full command list for this lab are in labs/lab-09-*/ . Run the commands from inside that folder.

Lab 10 — Build Resilience with Probes, Disruption Budgets and Multi-Tier Architecture

Objective: Prepare a technical blueprint for a resilient multi-tier cloud native architecture and prove its failure behaviour.

Goal: The learner deploys a three-tier application, configures liveness, readiness and startup probes, deliberately fails a container to watch Kubernetes self-heal, and protects availability during maintenance with a PodDisruptionBudget.

What you'll build

A three-tier application with health probes and a PodDisruptionBudget, plus a documented blueprint (Tools: liveness/readiness/startup probes, PodDisruptionBudget, Deployment, Service, resource limits.)

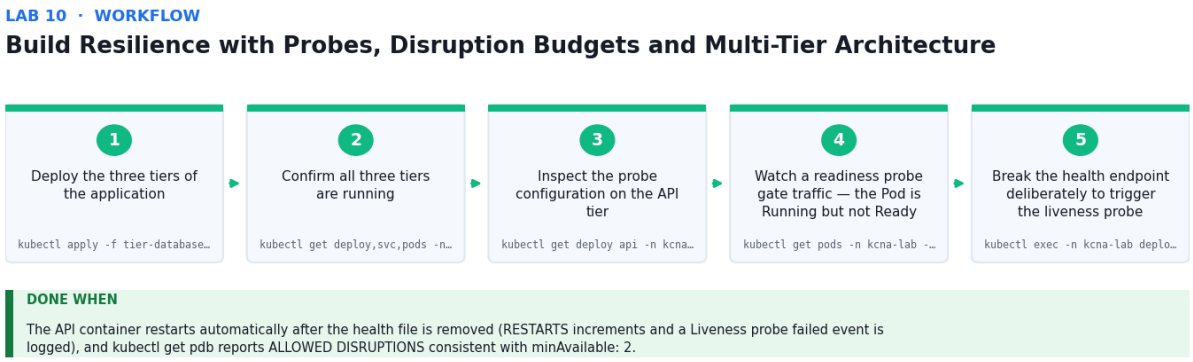


Figure 10: the Build Resilience with Probes, Disruption Budgets and Multi-Tier Architecture workflow — the first five stages and the verification criterion.

Step-by-step

1. Deploy the three tiers of the application

```
kubectl apply -f tier-database.yaml -f tier-api.yaml -f tier-
  frontend.yaml
```

2. Confirm all three tiers are running and their Services resolve

```
kubectl get deploy,svc,pods -n kcna-lab -l tier
```

3. Inspect the probe configuration on the API tier

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
kubectl get deploy api -n kcna-lab -o
  jsonpath='{.spec.template.spec.containers[0].livenessProbe}' | python3
  -m json.tool
```

4. Watch a readiness probe gate traffic — the Pod is Running but not Ready

```
kubectl get pods -n kcna-lab -l tier=api -o wide
```

5. Break the health endpoint deliberately to trigger the liveness probe

```
kubectl exec -n kcna-lab deploy/api -- rm -f
  /usr/share/nginx/html/healthz
```

6. Watch Kubernetes restart the unhealthy container automatically

```
kubectl get pods -n kcna-lab -l tier=api -w --timeout=120s
```

7. Confirm the restart count and the probe failure event

```
kubectl describe pod -n kcna-lab -l tier=api | grep -A5 'Liveness probe
  failed'
```

8. Apply a PodDisruptionBudget to protect availability during maintenance

```
kubectl apply -f pdb-api.yaml && kubectl get pdb -n kcna-lab
```

9. Attempt a node drain and observe the PDB blocking excessive eviction

```
kubectl drain minikube --ignore-daemonsets --delete-emptydir-data --dry-
  run=server 2>&1 | head -15
```

10. Complete blueprint-template.md documenting components, interfaces and NFRs

```
cat blueprint-template.md
```

Test it

The API container restarts automatically after the health file is removed (RESTARTS increments and a Liveness probe failed event is logged), and kubectl get pdb reports ALLOWED DISRUPTIONS consistent with minAvailable: 2.

Note: All manifests, mock data and the full command list for this lab are in labs/lab-10-*/ . Run the commands from inside that folder.

Lab 11 — Explore Serverless, Open Standards and the CNCF Landscape

Objective: Identify stakeholder demands and open standards that constrain a cloud native technical blueprint.

Goal: The learner examines the open standards that keep Kubernetes portable — OCI, CRI, CNI, CSI — by inspecting the cluster's own implementations, models event-driven scale-to-zero, and maps candidate CNCF projects to the blueprint's requirements.

What you'll build

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

A standards-and-projects selection matrix justifying each component of the blueprint (Tools: CNCF landscape, OCI, CRI, CNI, CSI, Knative and KEDA concepts, ConfigMap, scale-to-zero.)

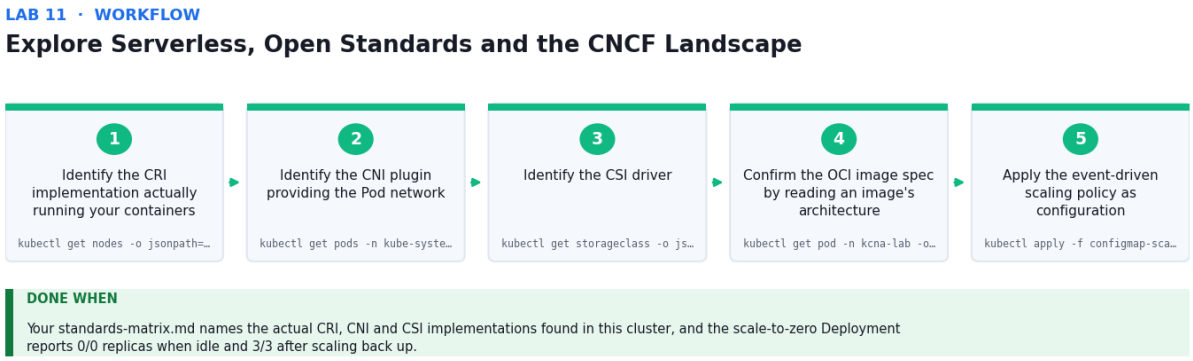


Figure 11: the Explore Serverless, Open Standards and the CNCF Landscape workflow — the first five stages and the verification criterion.

Step-by-step

1. Identify the CRI implementation actually running your containers

```
kubectl get nodes -o
  jsonpath='{.items[*].status.nodeInfo.containerRuntimeVersion}'
```

2. Identify the CNI plugin providing the Pod network

```
kubectl get pods -n kube-system -o name | grep -Ei 'cni|calico|flannel|
  kindnet'
```

3. Identify the CSI driver and StorageClass provisioner in use

```
kubectl get storageclass -o jsonpath='{.items[*].provisioner}'
```

4. Confirm the OCI image spec by reading an image's architecture and layers

```
kubectl get pod -n kcna-lab -o
  jsonpath='{.items[0].status.containerStatuses[0].imageID}'
```

5. Apply the event-driven scaling policy as configuration

```
kubectl apply -f configmap-scaling-policy.yaml
```

6. Deploy the workload that will demonstrate scale-to-zero

```
kubectl apply -f deployment-scale-to-zero.yaml
```

7. Scale the workload to zero to model the serverless idle state

```
kubectl scale deploy scale-to-zero -n kcna-lab --replicas=0
```

8. Confirm no Pods and therefore no compute cost while idle

```
kubectl get deploy scale-to-zero -n kcna-lab
```

9. Scale back up on demand and measure the cold-start latency

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
time kubectl scale deploy scale-to-zero -n kcna-lab --replicas=3 --
  timeout=60s
```

10. Complete cncf-selection.csv and standards-matrix.md for your blueprint

```
cat cncf-selection.csv
```

Test it

Your standards-matrix.md names the actual CRI, CNI and CSI implementations found in this cluster, and the scale-to-zero Deployment reports 0/0 replicas when idle and 3/3 after scaling back up.

Note: All manifests, mock data and the full command list for this lab are in labs/lab-11-*/ . Run the commands from inside that folder.

Topic 04 — Kubernetes Monitoring and Observability (8%)

Telemetry & Observability · Prometheus · Cost Management

Key concepts

- Monitoring answers questions you already knew to ask; observability lets you ask new questions of a running system without shipping new code.
- The three pillars are metrics (cheap, aggregatable numbers), logs (discrete events with detail) and traces (one request's path across services).
- Prometheus pulls metrics by scraping an HTTP /metrics endpoint on a schedule, and stores them as a time series identified by a name and a set of labels.
- The four Prometheus metric types: counter (only increases), gauge (goes up and down), histogram (bucketed observations) and summary (client-side quantiles).
- PromQL selects a series then applies functions — rate() over a counter gives per-second throughput; histogram_quantile() over buckets gives latency percentiles.
- The four golden signals — latency, traffic, errors and saturation — are the minimum viable dashboard for any service.
- Alert on symptoms that users feel, not on causes: page on the error-rate SLO burning, not on a single Pod restarting.
- OpenTelemetry is the vendor-neutral standard for generating and exporting metrics, logs and traces, so instrumentation survives a change of backend.
- Cost management is an observability problem: unbounded requests, idle nodes and over-provisioned limits are all visible in the same metrics.
- FinOps practices — right-sizing from real usage, cluster autoscaling, spot nodes and namespace-level showback — turn those metrics into savings.

Lab 12 — Instrument and Collect Metrics with Prometheus

Objective: Demonstrate how observability components collect the telemetry needed to address a business problem.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: The learner deploys Prometheus into the cluster, exposes an application /metrics endpoint carrying mock order and latency data, confirms the scrape targets are healthy, and queries the data with PromQL to answer a stated business question.

What you'll build

A running Prometheus scraping a custom application exporter, queried with PromQL (Tools: Prometheus, exporters, /metrics endpoint, PromQL, ServiceMonitor concepts, counters/gauges/histograms.)

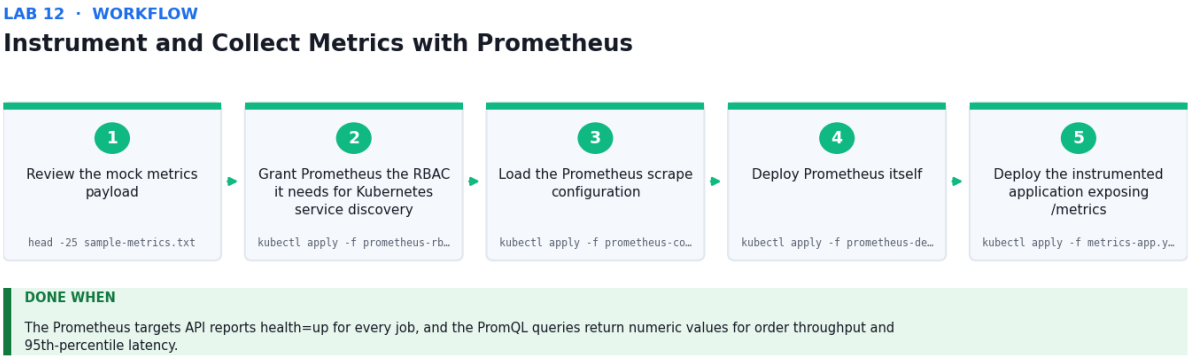


Figure 12: the Instrument and Collect Metrics with Prometheus workflow — the first five stages and the verification criterion.

Step-by-step

1. Review the mock metrics payload and identify each metric type

   ```
   head -25 sample-metrics.txt
   ```
2. Grant Prometheus the RBAC it needs for Kubernetes service discovery

   ```
   kubectl apply -f prometheus-rbac.yaml
   ```
3. Load the Prometheus scrape configuration

   ```
   kubectl apply -f prometheus-config.yaml
   ```
4. Deploy Prometheus itself

   ```
   kubectl apply -f prometheus-deployment.yaml
   ```
5. Deploy the instrumented application exposing /metrics

   ```
   kubectl apply -f metrics-app.yaml
   ```
6. Confirm the application is exposing metrics in the Prometheus text format

   ```
   kubectl exec -n kcna-lab deploy/metrics-app -- wget -qO-
     localhost:8080/metrics | head -15
   ```
7. Port-forward the Prometheus UI to your workstation

   ```
   kubectl port-forward -n kcna-lab svc/prometheus 9090:9090 &
   ```
8. Verify every scrape target is UP before trusting the data

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
curl -s 'http://localhost:9090/api/v1/targets' | python3 -c "import
  json,sys; [print(t['labels'].get('job'), t['health']) for t in
  json.load(sys.stdin)['data']['activeTargets']]"
```

9. Query per-second order throughput with rate() over the counter

```
curl -s --data-urlencode 'query=rate(orders_total[5m])'
  http://localhost:9090/api/v1/query
```

10. Compute the 95th-percentile latency from the histogram buckets

```
curl -s --data-urlencode 'query=histogram_quantile(0.95,
  rate(request_duration_seconds_bucket[5m]))'
  http://localhost:9090/api/v1/query
```

Test it

The Prometheus targets API reports health=up for every job, and the PromQL queries return numeric values for order throughput and 95th-percentile latency.

Note: All manifests, mock data and the full command list for this lab are in labs/lab-12-*/ . Run the commands from inside that folder.

Lab 13 — Apply the Golden Signals, Alerting and Cost Management

Objective: Demonstrate how observability and cost data together resolve a specific business problem.

Goal: The learner builds the four golden signals for the order service, writes an alerting rule that fires on the symptom rather than the cause, then analyses cost data — requests versus actual usage — and recommends right-sizing backed by evidence.

What you'll build

A golden-signals dashboard specification, a firing alert rule, and a costed right-sizing recommendation (Tools: Prometheus alerting rules, golden signals, kubectl top, resource requests, FinOps right-sizing.)

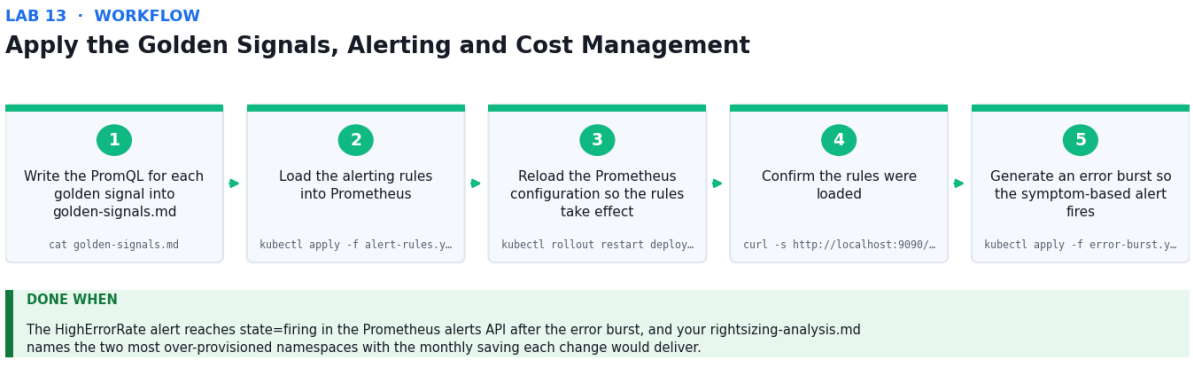


Figure 13: the Apply the Golden Signals, Alerting and Cost Management workflow — the first five stages and the verification criterion.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step-by-step

1. Write the PromQL for each golden signal into golden-signals.md

```
cat golden-signals.md
```

2. Load the alerting rules into Prometheus

```
kubectl apply -f alert-rules.yaml
```

3. Reload the Prometheus configuration so the rules take effect

```
kubectl rollout restart deploy/prometheus -n kcna-lab
```

4. Confirm the rules were loaded and are evaluating

```
curl -s http://localhost:9090/api/v1/rules | python3 -m json.tool | head
  -25
```

5. Generate an error burst so the symptom-based alert fires

```
kubectl apply -f error-burst.yaml
```

6. Watch the alert transition from Pending to Firing

```
curl -s http://localhost:9090/api/v1/alerts | python3 -m json.tool |
  head -25
```

7. Compare requested resources against what the Pods actually use

```
kubectl top pods -n kcna-lab --sort-by=cpu
```

8. Read the mock cost report and identify the two worst-value namespaces

```
column -s, -t cost-report.csv
```

9. Calculate the waste from over-requested CPU and memory

```
python3 -c "import csv;r=list(csv.DictReader(open('cost-report.csv')));
  [print(x['namespace'],
  round(1-float(x['cpu_used'])/float(x['cpu_requested']),2)) for x in
  r]"
```

10. Record your costed right-sizing recommendation with the evidence

```
cat rightsizing-analysis.md
```

Test it

The HighErrorRate alert reaches state=firing in the Prometheus alerts API after the error burst, and your rightsizing-analysis.md names the two most over-provisioned namespaces with the monthly saving each change would deliver.

Note: All manifests, mock data and the full command list for this lab are in labs/lab-13-*/ . Run the commands from inside that folder.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Topic 05 — Kubernetes Application Deployment and Management (8%)

Application Delivery Fundamentals · GitOps · CI/CD

Key concepts

- Application delivery on Kubernetes is the practice of moving a change from a commit to running Pods safely, repeatably and reversibly.
- A Deployment's RollingUpdate strategy with maxSurge and maxUnavailable is the primitive behind every zero-downtime release.
- Progressive delivery reduces blast radius: blue-green swaps traffic at once, canary shifts a small percentage first and watches the metrics.
- kubectl rollout status, history and undo are the operational loop — every rollout must be reversible before it is attempted.
- Packaging manifests matters: Helm templates a chart with values, while Kustomize overlays patches onto a plain base with no templating language.
- GitOps makes Git the single source of truth: the desired state is declared in a repository and an in-cluster agent continuously reconciles the cluster to it.
- Because reconciliation is continuous, GitOps also corrects configuration drift — a manual kubectl edit is reverted automatically.
- Argo CD and Flux are the two CNCF-graduated GitOps engines; both watch a repo, diff against the cluster and apply the difference.
- A CI/CD pipeline builds and tests the image, then updates the manifest repository — CI produces artifacts, CD reconciles state.
- A system review closes the loop: read the observability data against the SLOs, decide what to change, and adapt the architecture accordingly.

Lab 14 — Perform Zero-Downtime Rolling Updates and Canary Releases

Objective: Implement a controlled release and roll it back, monitoring solution status throughout.

Goal: The learner tunes a Deployment's rolling-update strategy, releases a new version while continuously probing for downtime, deliberately deploys a broken image to see the rollout stall, rolls back, then runs a weighted canary release.

What you'll build

A verified zero-downtime rolling update, a recovered failed rollout, and a working canary split (Tools: Deployment strategy, maxSurge/maxUnavailable, readiness probes, kubectl rollout, canary via replica weighting.)

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

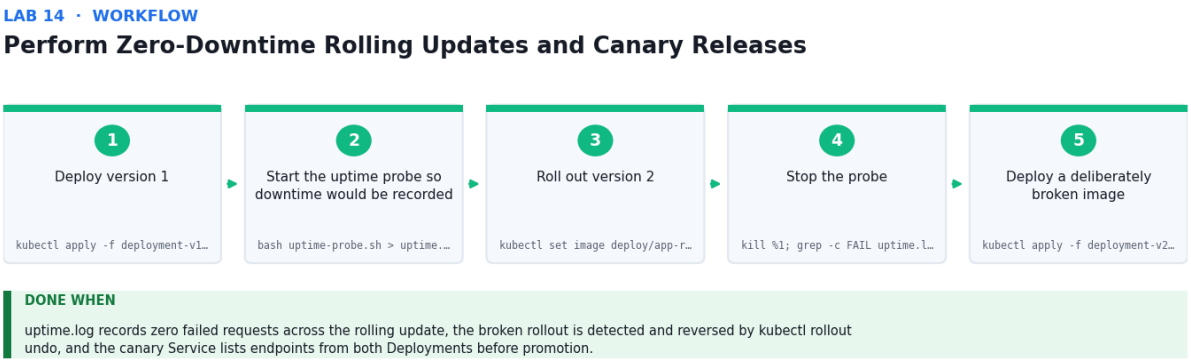


Figure 14: the Perform Zero-Downtime Rolling Updates and Canary Releases workflow — the first five stages and the verification criterion.

Step-by-step

1. Deploy version 1 and review its rolling-update strategy

```
kubectl apply -f deployment-v1.yaml && kubectl get deploy app-release -n
  kcna-lab -o jsonpath='{.spec.strategy}'
```

2. Start the uptime probe so downtime would be recorded

```
bash uptime-probe.sh > uptime.log 2>&1 &
```

3. Roll out version 2 and watch replicas cut over incrementally

```
kubectl set image deploy/app-release app=nginx:1.27-alpine -n kcna-lab
  && kubectl rollout status deploy/app-release -n kcna-lab
```

4. Stop the probe and confirm no requests failed during the release

```
kill %1; grep -c FAIL uptime.log || echo 'zero failures'
```

5. Deploy a deliberately broken image and watch the rollout stall

```
kubectl apply -f deployment-v2-broken.yaml
```

6. Diagnose why the rollout is stuck rather than guessing

```
kubectl rollout status deploy/app-release -n kcna-lab --timeout=60s;
  kubectl get pods -n kcna-lab -l app=app-release
```

7. Roll back to the last good revision and confirm recovery

```
kubectl rollout undo deploy/app-release -n kcna-lab && kubectl rollout
  status deploy/app-release -n kcna-lab
```

8. Deploy the stable and canary Deployments behind one Service

```
kubectl apply -f canary-stable.yaml -f canary-new.yaml -f service-
  canary.yaml
```

9. Confirm the Service is load-balancing across both versions

```
kubectl get endpoints app-canary -n kcna-lab -o
  jsonpath='{.subsets[0].addresses[*].ip}'
```

10. Measure the actual traffic split, then promote or abort the canary

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
kubectl scale deploy app-canary -n kcna-lab --replicas=10 && kubectl
  scale deploy app-stable -n kcna-lab --replicas=0
```

Test it

uptime.log records zero failed requests across the rolling update, the broken rollout is detected and reversed by kubectl rollout undo, and the canary Service lists endpoints from both Deployments before promotion.

Note: All manifests, mock data and the full command list for this lab are in labs/lab-14-*/ . Run the commands from inside that folder.

Lab 15 — Package Applications with Helm and Kustomize

Objective: Package and parameterise a solution so it can be delivered repeatably across environments.

Goal: The learner packages the application as a Helm chart with environment-specific values, installs and upgrades it, then achieves the same result with Kustomize overlays and compares the two approaches for the delivery blueprint.

What you'll build

An installed and upgraded Helm release plus equivalent dev and prod Kustomize overlays (Tools: Helm (chart, values, release, upgrade, rollback), Kustomize (base, overlays, patches), kubectl apply -k.)

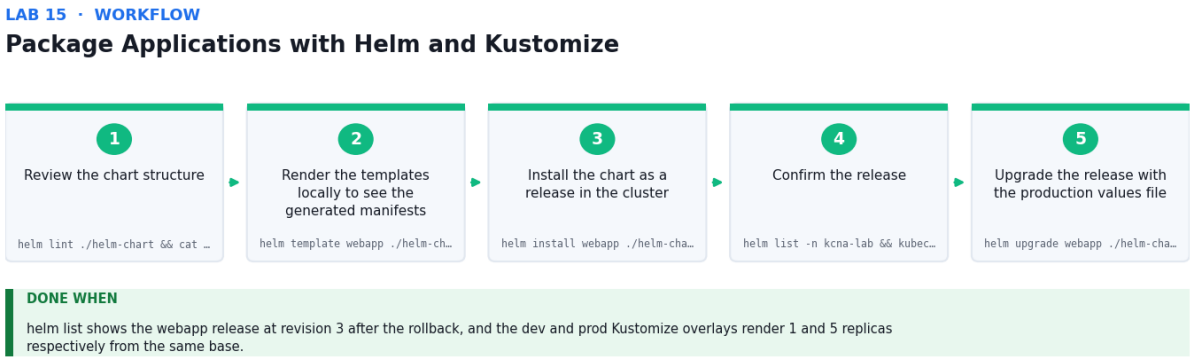


Figure 15: the Package Applications with Helm and Kustomize workflow — the first five stages and the verification criterion.

Step-by-step

1. Review the chart structure and where values are substituted

```
helm lint ./helm-chart && cat helm-chart/values.yaml
```

2. Render the templates locally to see the generated manifests

```
helm template webapp ./helm-chart | head -40
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

3. Install the chart as a release in the cluster

```
helm install webapp ./helm-chart --namespace kcna-lab
```

4. Confirm the release and the resources it created

```
helm list -n kcna-lab && kubectl get all -n kcna-lab -l
  app.kubernetes.io/instance=webapp
```

5. Upgrade the release with the production values file

```
helm upgrade webapp ./helm-chart -f helm-chart/values-prod.yaml -n kcna-
  lab
```

6. Verify the upgrade took effect and review the release history

```
kubectl get deploy -n kcna-lab -l app.kubernetes.io/instance=webapp &&
  helm history webapp -n kcna-lab
```

7. Roll the release back to revision 1

```
helm rollback webapp 1 -n kcna-lab
```

8. Build the dev overlay with Kustomize and inspect the output

```
kubectl kustomize kustomize/overlays/dev | head -30
```

9. Apply the prod overlay and compare it with the dev result

```
kubectl apply -k kustomize/overlays/prod
```

10. Compare the two approaches and record which suits your blueprint

```
diff <(kubectl kustomize kustomize/overlays/dev) <(kubectl kustomize
  kustomize/overlays/prod) | head -20
```

Test it

helm list shows the webapp release at revision 3 after the rollback, and the dev and prod Kustomize overlays render 1 and 5 replicas respectively from the same base.

Note: All manifests, mock data and the full command list for this lab are in labs/lab-15-*/ . Run the commands from inside that folder.

Lab 16 — Automate Delivery with GitOps and CI/CD, and Conduct a System Review

Objective: Implement regular system reviews to monitor solution status and adapt within a cloud native framework.

Goal: The learner models the GitOps reconciliation loop with a Git-backed manifest repository, deliberately introduces configuration drift to watch reconciliation correct it, walks a CI/CD pipeline definition, then conducts a structured system review against the SLOs and records the adaptations.

What you'll build

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

A reconciling GitOps repository that self-corrects drift, plus a completed system review report (Tools: GitOps, Argo CD / Flux concepts, Git, CI/CD pipeline, reconciliation loop, SLOs, system review.)

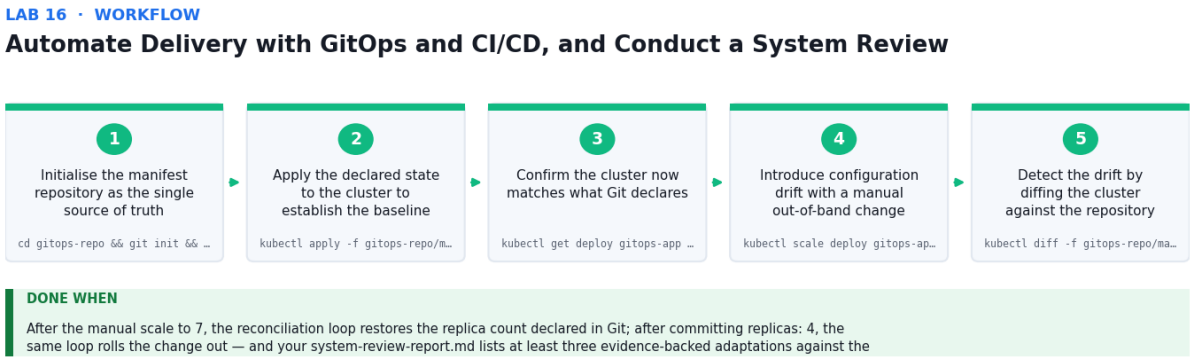


Figure 16: the Automate Delivery with GitOps and CI/CD, and Conduct a System Review workflow — the first five stages and the verification criterion.

Step-by-step

1. Initialise the manifest repository as the single source of truth

```
cd gitops-repo && git init && git add . && git commit -m 'Declare
  desired state'
```

2. Apply the declared state to the cluster to establish the baseline

```
kubectl apply -f gitops-repo/manifests/
```

3. Confirm the cluster now matches what Git declares

```
kubectl get deploy gitops-app -n kcna-lab -o jsonpath='{.spec.replicas}'
```

4. Introduce configuration drift with a manual out-of-band change

```
kubectl scale deploy gitops-app -n kcna-lab --replicas=7
```

5. Detect the drift by diffing the cluster against the repository

```
kubectl diff -f gitops-repo/manifests/deployment.yaml || true
```

6. Run the reconciliation loop and watch the drift be corrected

```
bash reconcile.sh
```

7. Confirm the cluster was returned to the declared state automatically

```
kubectl get deploy gitops-app -n kcna-lab -o jsonpath='{.spec.replicas}'
```

8. Change the desired state in Git and let reconciliation roll it out

```
cd gitops-repo && sed -i.bak 's/replicas: 3/replicas: 4/'
  manifests/deployment.yaml && git commit -am 'Scale to 4' && cd .. &&
  bash reconcile.sh
```

9. Walk the CI/CD pipeline and identify which stage is CI and which is CD

```
cat pipeline.yaml
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

10. Conduct the system review against the SLOs and record the adaptations

```
column -s, -t slo-targets.csv && cat system-review-report.md
```

Test it

After the manual scale to 7, the reconciliation loop restores the replica count declared in Git; after committing replicas: 4, the same loop rolls the change out — and your system-review-report.md lists at least three evidence-backed adaptations against the breached SLOs.

Note: All manifests, mock data and the full command list for this lab are in labs/lab-16-*/ . Run the commands from inside that folder.

Exam Focus — Cross-Cutting Topics

These topics are examined across the five KCNA domains but do not each have a dedicated hands-on lab. Study this section alongside the labs.

Cloud native roles and personas

- Platform engineer — builds and runs the internal platform other teams deploy onto; owns the cluster, not the applications.
- Site Reliability Engineer (SRE) — owns reliability through SLIs, SLOs and error budgets; decides when to stop shipping and fix.
- DevOps engineer — automates the path from commit to production, and the feedback loop back to developers.
- Security engineer — owns the 4C model in practice: cloud posture, cluster hardening, container supply chain and code.
- Application developer — packages the workload, declares its resources and health checks, and reads its own telemetry.

Community, governance and the CNCF

- The CNCF hosts Kubernetes and hundreds of related projects under vendor-neutral governance.
- Project maturity runs Sandbox (early, experimental) → Incubating (proven production adopters) → Graduated (mature, widely deployed).
- Graduated projects include Kubernetes, Prometheus, Envoy, containerd, Helm, Argo, Fluentd, etcd, Linkerd and OpenTelemetry.
- Kubernetes itself is developed by Special Interest Groups (SIGs); changes go through the Kubernetes Enhancement Proposal (KEP) process.
- Contribution is open — the maturity level is a genuine signal to use when justifying a component in a technical blueprint.

Open standards that prevent lock-in

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- OCI (Open Container Initiative) — the image and runtime specifications, so an image built with Docker runs under containerd.
- CRI (Container Runtime Interface) — lets Kubernetes drive containerd or CRI-O interchangeably.
- CNI (Container Network Interface) — the plugin contract for Pod networking (Calico, Cilium, Flannel).
- CSI (Container Storage Interface) — the plugin contract for storage, so a PVC works the same on any backend.
- OpenTelemetry — vendor-neutral generation and export of metrics, logs and traces, so instrumentation outlives your backend choice.
- SMI (Service Mesh Interface) — a common surface across service-mesh implementations.

Serverless and event-driven scaling

- Serverless removes server management from the developer's concern and bills only for what runs.
- Knative brings request-driven, scale-to-zero serving onto Kubernetes itself.
- KEDA scales workloads on event sources — queue depth, topic lag, a cron schedule — not just CPU.
- The trade-off is cold-start latency: scaling to zero saves cost, but the first request after idle pays to start a Pod.

Revision Tips

- First pass: complete every lab in order, reading the README in each lab folder.
- Second pass: redo each lab from an empty folder, writing the manifests yourself until it is automatic.
- Review the 'Test it' check for every lab.
- Re-read the Key Concepts for each topic.
- Weight your revision by exam weighting: Topic 1 is 46% of the KCNA exam, Topic 2 is 22%.

Glossary

- **Pod —** The smallest deployable unit in Kubernetes — one or more containers sharing a network namespace, IP address and volumes.
- **Node —** A worker machine (virtual or physical) that runs Pods, managed by the kubelet.
- **Control plane —** The components that decide what should run: kube-apiserver, etcd, kube-scheduler and kube-controller-manager.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- **etcd —** The consistent, highly available key-value store holding the entire cluster state.
- **kubelet —** The agent on each node that watches the API server and drives the container runtime to start Pods.
- **kube-proxy —** The component that programmes iptables/IPVS rules so a Service virtual IP reaches a healthy Pod.
- **Deployment —** The controller that manages ReplicaSets to provide scaling, rolling updates and rollback for stateless apps.
- **ReplicaSet —** The controller that keeps exactly N identical Pods running; managed for you by a Deployment.
- **StatefulSet —** The controller giving each replica a stable network identity and its own persistent volume.
- **DaemonSet —** The controller that runs exactly one Pod on every matching node — used for agents.
- **Service —** A stable virtual IP and DNS name in front of an ephemeral, label-selected set of Pods.
- **Ingress —** An L7 HTTP(S) entry point that routes by host and path to Services, and terminates TLS.
- **NetworkPolicy —** A Pod-level firewall; selecting a Pod with a policy switches it from default-allow to default-deny.
- **ConfigMap / Secret —** Objects holding configuration data — Secrets are base64-encoded and access-controlled, but not encrypted by default.
- **RBAC —** Role-Based Access Control — binds a subject to a Role (namespaced) or ClusterRole (cluster-wide).
- **ServiceAccount —** The identity a Pod runs as when it talks to the Kubernetes API.
- **PersistentVolumeClaim (PVC) —** A request for storage; a StorageClass provisions the PersistentVolume that satisfies it.
- **StorageClass —** An object describing a class of storage and the provisioner that creates it dynamically.
- **HPA —** Horizontal Pod Autoscaler — adds or removes Pod replicas based on observed metrics.
- **Probe —** A health check: liveness restarts a hung container, readiness gates traffic, startup protects slow boots.
- **PodDisruptionBudget —** A policy limiting how many Pods of an application may be voluntarily evicted at once.
- **Observability —** The ability to ask new questions of a running system from its metrics, logs and traces.
- **Prometheus —** The CNCF metrics system that scrapes /metrics endpoints and stores labelled time series.
- **PromQL —** Prometheus' query language — rate() over counters, histogram_quantile() over buckets.
- **Golden signals —** Latency, traffic, errors and saturation — the minimum viable dashboard for any service.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- **Helm —** A package manager for Kubernetes; a chart is templated with values to produce a release.
- **Kustomize —** Template-free customisation — patches overlaid onto a plain base, built into kubectl.
- **GitOps —** An operating model where Git holds the desired state and an in-cluster agent continuously reconciles to it.
- **Reconciliation loop —** The continuous compare-and-correct cycle that makes actual state match desired state.
- **CNCF —** The Cloud Native Computing Foundation — the vendor-neutral home of Kubernetes and the wider ecosystem.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg